Testing Documentation

1. Testing Overview

Objective

The objective of testing is to ensure that the Expense Tracker application works correctly, provides a smooth user experience, and handles invalid input gracefully.

Testing Type

- Manual Testing
- Functional Testing
- UI Testing
- Navigation Testing
- Validation Testing
- Future API Integration Testing

---

2. UI Testing

Each screen should be checked against the approved Figma design.

The following should be verified:

- Screen layout
- Colors
- Typography
- Spacing
- Icons
- Images
- Buttons
- Cards
- Scroll behavior
- Bottom tab navigation
- Different screen sizes

---

3. Form Testing

Forms should be tested using both valid and invalid input.

Examples:

Test Case	Expected Result
Empty required field	Validation error displayed
Invalid email	Validation error displayed
Valid email	Accepted
Password mismatch	Validation error displayed
Matching passwords	Accepted
Valid profile information	Saved successfully

---

4. Transaction Testing

Test the following:

- Income transaction appears correctly.
- Expense transaction appears correctly.
- Correct transaction details are displayed.
- Income and expense are distinguished correctly.
- Transaction Details opens with the correct data.
- Amounts, dates, fees, and totals are displayed correctly.

---

5. Profile Testing

Verify that:

- Profile information is displayed correctly.
- Profile menu items are displayed according to the Figma design.
- Account info opens Edit Profile.
- Login and Security opens Change Password.
- Edit Profile fields can be modified.
- Save Changes returns to the Profile screen.
- Change Password validates the entered information.
- Password confirmation must match.

---

6. API Testing

After Django backend integration, API requests should be tested for:

- Successful requests
- Invalid requests
- Missing required fields
- Unauthorized requests
- Expired authentication
- Server errors
- Network failures
- Invalid response data

---

7. Regression Testing

Whenever a new feature is added or an existing feature is modified, previously implemented functionality should be tested again to ensure that the change has not introduced new problems.

New Feature
↓
Implement
↓
Test New Feature
↓
Run Existing Feature Tests
↓
Any Regression?
↙ ↘
Yes No
↓ ↓
Fix Issue Continue

---

8. Test Environment

Item	Value
Platform	Android
IDE	Visual Studio Code
Framework	React Native

Item	Value
Language	TypeScript
Operating System	Windows 11
Device	Android Emulator / Physical Android Device

---

9. Login Module Test Cases

Test ID	Test Scenario	Input	Expected Result	Status
TC-001	Valid Login	Correct Email & Password	Navigate to Home Screen	Pass
TC-002	Empty Email	Blank Email	Validation Message	Pass
TC-003	Empty Password	Blank Password	Validation Message	Pass
TC-004	Invalid Email Format	abc@gmail	Show Invalid Email Error	Pass
TC-005	Wrong Credentials	Incorrect Password	Login Failed Message	Pending (Backend)

---

10. Signup Module Test Cases

Test ID	Test Scenario	Expected Result
TC-006	All Fields Valid	User Registered
TC-007	Empty Fields	Validation Error

Test ID	Test Scenario	Expected Result
TC-008	Passwords Do Not Match	Error Message
TC-009	Invalid Email	Validation Error

---

11. Forgot Password Test Cases

Test ID	Scenario	Expected Result
TC-010	Registered Email	Reset Link Sent
TC-011	Empty Email	Validation Error
TC-012	Invalid Email	Validation Error

---

12. Home Screen Test Cases

Test ID	Scenario	Expected Result
TC-013	Dashboard Opens	Balance Displayed
TC-014	Add Income Button	Navigate to Add Income
TC-015	Add Expense Button	Navigate to Add Expense
TC-016	See All Button	Open Transaction History

---

13. Statistics Screen Test Cases

Test ID	Scenario	Expected Result
TC-017	Week Tab	Week Data Displayed
TC-018	Month Tab	Month Data Displayed
TC-019	Year Tab	Year Data Displayed
TC-020	Category List	Categories Displayed

---

14. Wallet Screen Test Cases

Test ID	Scenario	Expected Result
TC-021	Wallet Screen Opens	Balance Displayed
TC-022	Add Button	Navigate to Add Money
TC-023	Pay Button	Navigate to Payment
TC-024	Send Button	Navigate to Transfer

---

15. Connect Wallet Test Cases

Test ID	Scenario	Expected Result
TC-025	Valid Card Details	Proceed to Next Screen
TC-026	Invalid Card Number	Validation Error
TC-027	Empty Fields	Validation Error

---

16. Navigation Test Cases

Test ID	Scenario	Expected Result
TC-028	Splash → Onboarding	Success
TC-029	Onboarding → Login	Success
TC-030	Login → Home	Success
TC-031	Home → Statistics	Success
TC-032	Home → Wallet	Success
TC-033	Wallet → Connect Wallet	Success

---

17. UI Testing Checklist

Verify the following:

- All buttons are clickable.
- All icons are visible.
- Text is readable.
- Colors match the Figma design.
- No overlapping components.
- Images load correctly.
- ScrollView works smoothly.
- Bottom Tab Navigation works correctly.
- SafeAreaView prevents content clipping.
- Keyboard does not hide input fields.

---

18. Performance Testing

The application should:

- Launch within 3 seconds.
- Navigate smoothly between screens.
- Scroll without lag.
- Avoid unnecessary re-rendering.
- Maintain memory efficiency.

---

19. Future Integration Testing

After integrating the Django backend, verify:

- User authentication.
- JWT token storage.
- API request and response handling.
- Network error handling.
- Database CRUD operations.

- Wallet connection flow.
- Statistics data synchronization.

---

20. Testing Status

At the current stage, testing is primarily focused on:

- UI implementation
- Navigation
- Form validation
- Screen transitions
- Visual comparison with Figma

Full API, database, authentication, and backend testing will be performed after Django backend integration.

---

21. Bug Report Template

Use this format to document issues during testing:

Field	Description
Bug ID	BUG-001
Module	Login
Description	Login button not responding
Severity	High
Priority	High
Status	Open / Fixed
Assigned To	Developer
Date	DD/MM/YYYY